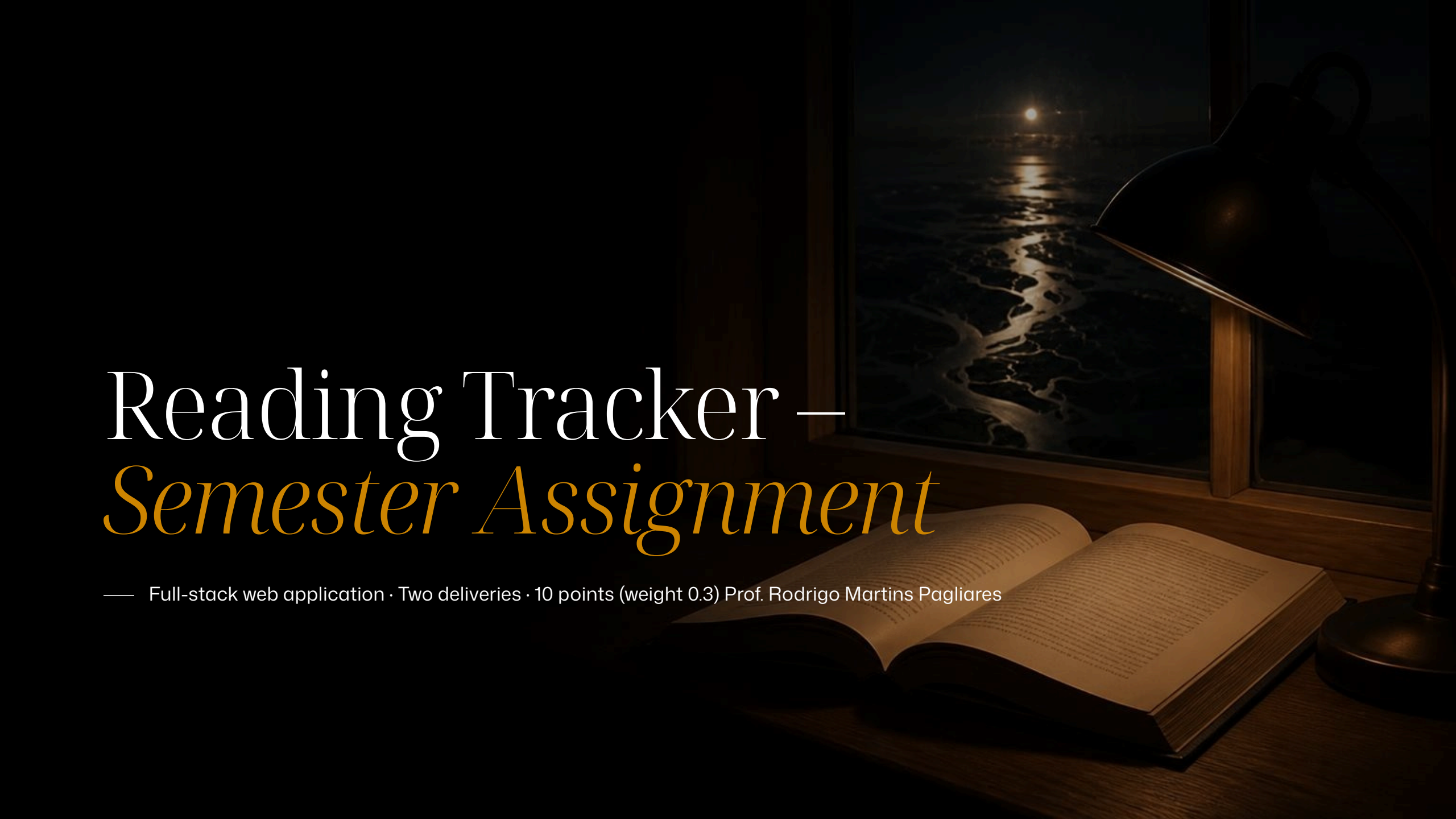


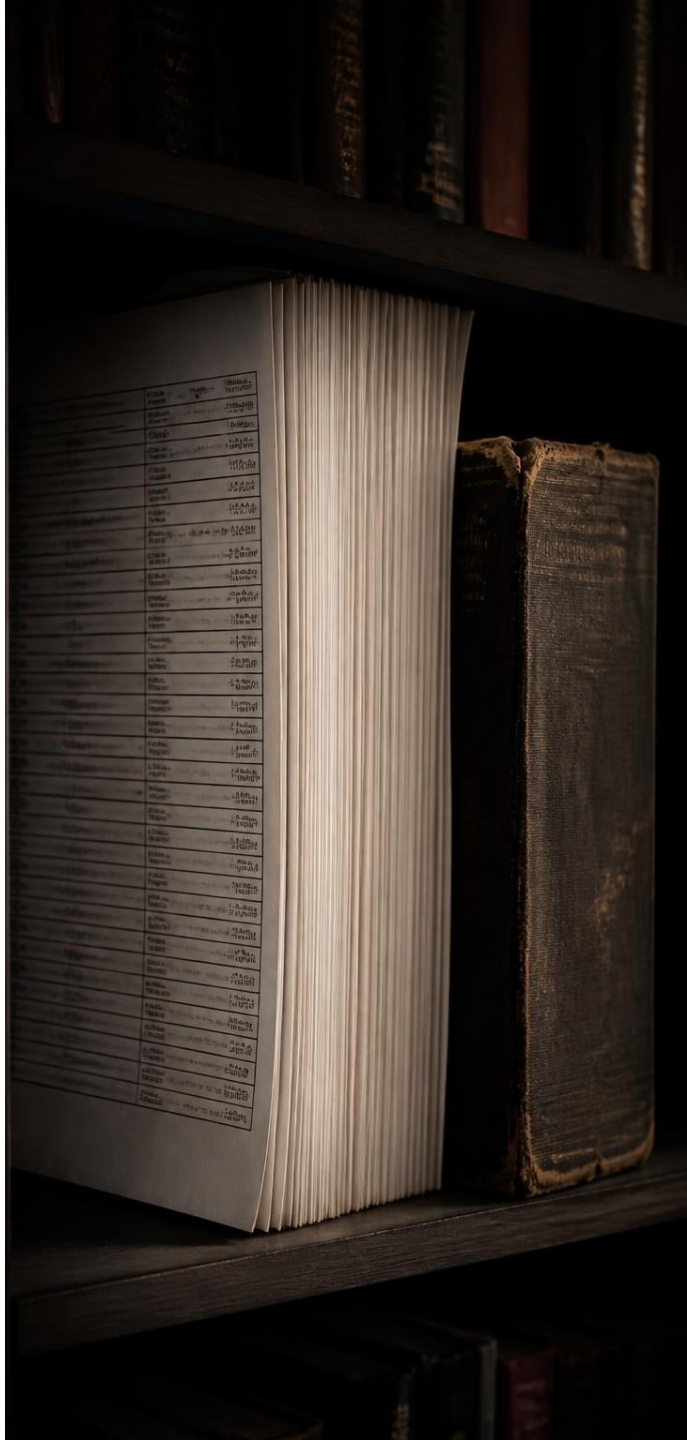
Reading Tracker — *Semester Assignment*

Full-stack web application · Two deliveries · 10 points (weight 0.3) Prof. Rodrigo Martins Pagliares

The problem: a reading log in a Word document

A reader tracks books read and books to read in a Word file. Your requirements source is How_Books_Read_Currently_Tracked.pdf.

- Inconsistent structure: recent years have year, form, mode, content and theme; older years only title, author and grade
- Mixed languages and values: "Romance" vs "Novel", "Prosa" vs "Prose", grades as "7", "7,0", "7.5", "7,5"
- One author, many spellings: Dostoevsky / Dostoiévski, Chekhov / Tchekhov
- Typos: "Ray Bradburry", "2001 – A Space Oddity"



More problems in the current log

Hand-maintained views fall out of date: books by century and authors by nationality are incomplete, misnumbered and inconsistently classified

The wishlist is mixed into the reading log, often without author, year or grade

Many entries lack a grade, year or classification, and nothing shows what still needs attention

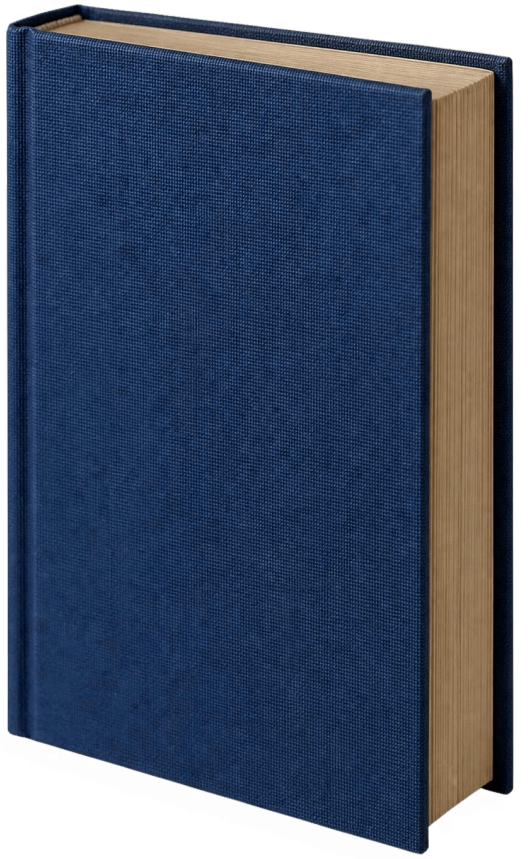
What you will build

A web application that replaces the document with:

Structured data

Consistent, validated input

Views computed automatically, never typed by hand



Interface and controlled values in US English. Titles and author names stay as recorded, in their original language. Keep interface text in one place so it could be translated later.

Two deliveries

Delivery 2 reuses Delivery 1: design for it from day one.

Delivery	Due (Brasília time)	Scope	Points
1 – Frontend	Nov 20, 2026, 23:59	Vanilla HTML, CSS, JS; data in the browser	5.0
2 – Full stack	Dec 11, 2026, 23:59	Spring Boot REST API, persistence, Delivery 1 frontend integrated	5.0

Logistics

- Teams: individual or up to 5 students; same requirements for every size
- Team composition goes in the Delivery 1 README and cannot change
- Public GitHub repository for both deliveries
- Tag the commit you want graded (delivery-1, delivery-2) and push the tag
- Late policy: up to 24 h late = 20% penalty; after 24 h, nothing is accepted

Technology stack

Frontend

- Required: vanilla HTML, CSS, JavaScript; fetch
- Allowed: Chart.js or similar, Google Fonts, icon fonts
- Not allowed: React, Vue, Angular, Svelte, jQuery, Bootstrap, Tailwind, TypeScript, npm, Vite, Webpack

Backend

- Required: Java 21+, Spring Boot 4.x, Maven or Gradle, REST with JSON
- Allowed: Spring Web, Spring Data JPA, Bean Validation, Lombok, DB driver (e.g., H2), DevTools, test starter
- Not allowed: Thymeleaf, JSP

— Any other library needs the instructor's approval by email, recorded in the README.

Domain model

Three main records plus two simple lists:

Author – a person who writes books

Book – the work itself

Reading – one time the person read, or plans to read, a book

Theme and **Country** – simple records with an ID and a name

Why separate Book and Reading? A book can be read more than once, and the wishlist and the reading log live in the same place.



Author and Book rules

Author

- Name required, unique ignoring case
- Nationality: one country from a fixed list (from the seed; cannot be edited)
- Birth date not in the future; death date after birth date

Book

- Title required; everything else optional (a book can be saved with only its title)
- Authors selected from existing records, never retyped
- Publication year: not in the future; negative = BC; year 0 is invalid
- Form, Mode, Content chosen from lists; Themes chosen or created on the spot, no duplicates

Controlled values

Form: Novel, Novella, Short Story, Short Story Collection, Play, Essay, Treatise, Diary, Graphic Novel

Mode: Prose, Drama, Poetry, Sequential Art

Content: Fiction, Non-fiction

Status: Want to read, Reading, Paused, Read, Abandoned

Priority: Low, Medium, High

— These are the minimum. You may add values, but each must have one consistent spelling.

Reading rules

- Book: required; a book can have several readings
- Status: required
- Year read: required when Read, optional when Abandoned, empty otherwise; not in the future, not before publication year
- Grade: 0.0 to 10.0 in steps of 0.5; only for Read or Abandoned
- Priority: only for Want to read
- Recommended by and Notes: free text

Core requirements (1/2)

FR-01 Manage authors – CRUD; an author linked to a book cannot be deleted, and the message names the linked books

---

FR-02 Manage books – CRUD; authors and themes selected from existing records, new theme on the spot

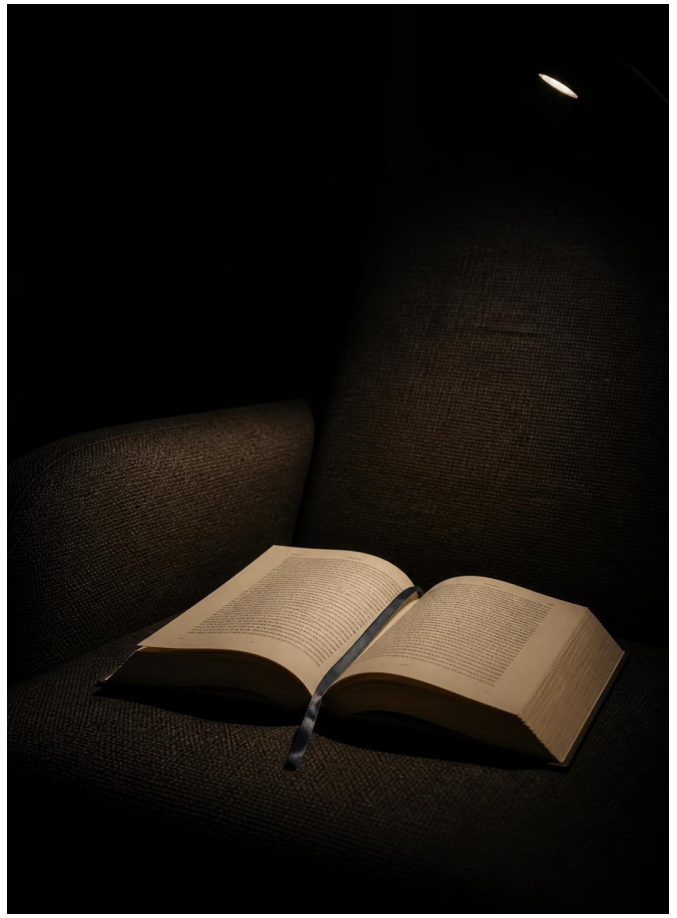
---

FR-03 Manage readings – add, edit, delete readings; the book form has an optional first-reading section, so one Save adds a book to the wishlist or logs it as read; deleting a book deletes its readings after a confirmation that says how many

FR-04 Status workflow

Want to read ⇄ Reading ⇄ Paused ⇄ Read ⇄ Abandoned

- Paused = started, will resume later
- Moving to Read requires a year read
- Leaving Want to read keeps recommendation and notes, clears priority
- Moving to a status that does not allow year read or grade clears them after the user confirms
- Re-reading a book = add a new reading, don't change the old one



Core requirements (2/2)

FR-05 Reading log – title, authors, status, year read, grade; sortable by each column; missing values shown as "-" and sorted last; default order: year read (newest first), then ID

FR-06 Validation – every Section 4 rule enforced in the form, message next to the failing field, invalid data never saved

FR-07 Seed data – loaded on first use only; user changes persist; Reset data (with confirmation) restores the seed

Delivery 1 – Data in the browser

- Put seed.json in your frontend folder (e.g., data/seed.json)
- First load: fetch it and save it in localStorage; later loads read only localStorage
- New records get the next integer ID after the highest existing one
- Reset data clears your keys and reloads seed.json
- fetch does not work from file:// – serve the folder (e.g., VS Code Live Server)

Delivery 1 – Prepare for Delivery 2

All data access in one module, e.g., store.js: getBooks(), saveBook(book), deleteBook(id)…

Every store function is async and called with await

Use the seed.json data shapes – they are also your API's shapes

---

On failure, throw an error shaped like the API's 400 response (each invalid field and the reason)

In Delivery 2 you rewrite only store.js

Delivery 1 – Code organization

- Separate HTML, CSS and JS files; JS split by responsibility: data access, rendering, validation (and charts)
- No inline onclick or style attributes, no style blocks in HTML; use addEventListener
- Two ways to connect JS files, graded equally:
  - Plain scripts with defer, in dependency order, one global scope
  - ES modules (recommended): export / import, one entry file with type="module"
- Must work on laptop and phone

Delivery 2 – REST API (minimum)

Method	Path	Purpose
GET, POST	/api/authors	List, create authors
GET, PUT, DELETE	/api/authors/{id}	Read, update, delete an author
GET, POST	/api/books	List, create books
GET, PUT, DELETE	/api/books/{id}	Read, update, delete a book (and its readings)
GET, POST	/api/books/{id}/readings	List, add a book's readings
GET	/api/readings	All readings (reading log)
PUT, DELETE	/api/readings/{id}	Update, delete a reading
GET, POST	/api/themes	List, create themes
GET	/api/countries	Fixed country list
POST	/api/reset	Delete all data, reload the seed

Status codes and errors

- Use 200, 201, 204, 400, 404, 409 appropriately
- 409: deleting an author linked to books; duplicate author or theme name
- All Section 4 rules checked again on the server
- One error format for 400, 404 and 409:

{ "status": 400, "message": "Validation failed", "errors": [{ "field": "yearRead", "message": "Year read is required when status is Read" }] }

- Layers: controllers (HTTP) → services (rules) → repositories (data). No business rules in controllers.

Persistence – recommended path

1. start.spring.io: Spring Web, Spring Data JPA, H2, Validation
2. File-based H2: jdbc:h2:file:./data/readingtracker, ddl-auto=update; add data/ to .gitignore
3. @Entity classes and one JpaRepository per entity
4. CommandLineRunner loads seed.json only when the DB is empty
    - Jackson 3 on Spring Boot 4: import from tools.jackson.databind
    - Let the DB generate IDs; insert countries → themes → authors → books → readings, mapping seed IDs to saved entities
    - Reuse the same method for POST /api/reset



IDs after a reset

Delivery 1: reset restores the seed exactly, IDs restart at 1

Delivery 2: the database never reuses IDs, so records come back with new, higher IDs

This is expected. Never hard-code IDs; always use the IDs the API returns

To start from 1 again: stop the app, delete data/, start again

Running the project

- One command: ./mvnw spring-boot:run (mvnw.cmd on Windows), or the Gradle equivalent
- Only Java 21+ needed; commit mvnw, mvnw.cmd and .mvn/
- Frontend in src/main/resources/static, opened at http://localhost:8080
- Relative URLs in store.js: /api/books, not http://localhost:8080/api/books
- Optional: H2 console via spring-boot-h2console

Seed data

- 143 readings, 2021–2026: 115 Read, 12 Paused, 16 Want to read
- Plus 143 books, 101 authors, 132 themes, 27 countries
- Already cleaned: one spelling per author, typos fixed, controlled values standardized
- Incomplete on purpose: books with no authors or year, authors without dates, no form/mode/theme before 2024
- Every record satisfies the rules: your validation must accept all of them

Grading – Delivery 1 (5.0)

Partially implemented = up to half the points.

Criterion	Points
FR-01 Manage authors	0.4
FR-02 Manage books	0.5
FR-03 Manage readings	0.5
FR-04 Status workflow	0.4
FR-05 Reading log	0.4
FR-06 Validation	0.5
FR-07 Seed data and reset	0.3
Code and interface quality	1.0
README	1.0

Grading – Delivery 2 (5.0)

- Project doesn't start following the README → at most 50%
- Forbidden framework or library → lose the dependent criteria

Criterion	Points
REST API	1.5
Persistence	1.0
Frontend integration	1.0
Server-side validation and errors	0.5
Layered architecture	0.5
README and video	0.5

What to submit

Email to pagliares@bcc.unifal-mg.edu.br, subject "Reading Tracker – Delivery N – [names]": repository link, self-assessment (0.0–5.0), video link (Delivery 2)

README: team members and IDs, how to run (with Java version), FR checklist, AI usage, endpoint list with examples and video link (Delivery 2)

Video (Delivery 2, max 15 min): demo of every requirement including persistence after restart, then a code walkthrough

— Commit throughout; each member's contribution should be visible in the history.

AI usage and academic integrity

- AI tools are allowed if disclosed in the README: tool, what for, which parts
- You must understand everything you submit: anyone may be asked to explain any code; failing costs 40% of that delivery
- Undisclosed AI use or copying from other teams = academic dishonesty
- Code from tutorials or forums is fine with a source comment
- AI-generated code must still respect the tech stack rules

Optional exercises (not graded)

- FR-08 Filters and search · FR-09 Wishlist view
- FR-10 Books by century · FR-11 Authors by nationality
- FR-12 Statistics · FR-13 Charts
- FR-14 Open Library autofill · FR-15 Import/export
- FR-16 Users and login · FR-17 Docker database · FR-18 DTOs and global error handling

---

Views must be computed from the data and handle gaps: "Unknown century", "Unknown" nationality, averages over graded readings only.

Next steps

- Read the statement and the original PDF log
- Form your team and create the repository
- Load seed.json and build store.js first
- Delivery 1: November 20 · Delivery 2: December 11

Questions?

